

BloodBridge: Optimizing Lifesaving Resources using AWS EC2 and RDS

1. PROJECT OVERVIEW

BloodBridge is a cloud-based full-stack web application designed to improve the efficiency and reliability of blood donation and emergency blood management systems. In real-world healthcare environments, one of the major challenges is the unavailability of the required blood type during emergencies and the delay in connecting hospitals with suitable donors. BloodBridge addresses this critical issue by providing a centralized digital platform that connects donors, hospitals, and blood banks in real time.

The system is built using modern web technologies including Python Flask for backend development, HTML/CSS/JavaScript with Bootstrap for frontend design, and MySQL for database management. The application is deployed on AWS cloud infrastructure using EC2 for hosting the web server and RDS for managing the database, ensuring scalability, availability, and security.

BloodBridge operates through three main user roles: donors, hospitals, and administrators. Donors can register on the platform, update their personal and health details such as blood group, location, and last donation date, and check their eligibility to donate blood. They can also respond to nearby blood requests and schedule donations conveniently. Hospitals can create emergency blood requests specifying blood type, quantity, urgency level, and location. The system then automatically matches these requests with suitable donors in the same region. Administrators manage the overall system, including monitoring users, tracking blood requests, and updating blood inventory.

A key feature of BloodBridge is its real-time matching and notification system, which ensures that emergency requests are quickly communicated to potential donors, reducing response time significantly. The system also includes a dynamic dashboard that displays live blood inventory updates, request statuses, and donor activity.

From a technical perspective, BloodBridge follows a client-server architecture where the frontend communicates with the Flask backend, which in turn interacts with the MySQL database hosted on

AWS RDS. The use of AWS EC2 ensures that the application remains accessible over the internet, while RDS provides secure and managed database storage.

Overall, BloodBridge is designed to enhance coordination between healthcare providers and donors, minimize delays in critical situations, and ensure better utilization of blood resources. It demonstrates the practical application of cloud computing, full-stack development, and real-time data processing in solving a life-saving real-world problem.

Project Member:

Darshana Ajit Deshmukh
(Data Science - final year)

2. OBJECTIVES

1. To develop a centralized blood donation management system
2. To reduce time delay in emergency blood requests
3. To connect donors, hospitals, and blood banks efficiently
4. To maintain real-time blood inventory updates
5. To deploy a scalable cloud-based web application using AWS

3. KEY FEATURES

1. Donor Registration and Login System:

Enables donors to securely register and log into the platform using authentication. It stores personal and blood group details for effective matching and tracking.

2. Hospital Emergency Blood Request Module:

Allows hospitals to create urgent blood requests by specifying blood type, quantity, and location. The system instantly notifies matching donors to speed up response time.

3. Blood Inventory Management System:

Maintains real-time tracking of available blood units in the blood bank. It automatically updates stock levels after donations and requests.

4. Donor Availability and Eligibility Tracking:

Checks whether a donor is eligible to donate based on last donation date and health criteria. It ensures safe and timely donation scheduling.

5. Real-Time Donor-Request Matching:

Automatically matches hospital requests with nearby eligible donors using blood group and location filters. It reduces search time during emergencies.

6. Interactive Dashboard with Notifications:

Provides a user-friendly dashboard showing requests, responses, and updates. Real-time notifications alert users about urgent activities.

7. Role-Based Access (Admin, Donor, Hospital):

Defines separate access levels for Admin, Donor, and Hospital users. Each role has specific permissions and features for secure system operation.

8. AWS Cloud Deployment (EC2 + RDS Integration):

Hosts the application on AWS EC2 while using RDS for database management. It ensures scalability, reliability, and cloud-based accessibility.

4. USE CASE SCENARIOS

Scenario 1: Emergency Blood Request

A hospital urgently requires O+ blood. The system automatically notifies nearby eligible donors and allows them to respond instantly.

Scenario 2: Donor Management

A donor logs into the system, checks eligibility, and schedules a donation based on nearby hospital requests.

Scenario 3: Blood Bank Inventory Update

A blood bank updates stock levels, and the system automatically reflects updated availability across all dashboards.

5. TECHNICAL APPROACH

1. Frontend: HTML, CSS, JavaScript, Bootstrap
2. Backend: Python Flask
3. Database: MySQL (AWS RDS)
4. Hosting: AWS EC2
5. Architecture: Client → Flask Server → RDS Database
6. Authentication: Session-based login system
7. API Communication: Flask routes handling CRUD operations

6. IMPLEMENTATION PLAN

Phase No.	Phase Name	Activities Included	Duration
1	Requirement Analysis	Study of problem, scope definition	1 Week
2	System Design	UI/UX design, database schema design	1 Week
3	Backend Development	Flask APIs, authentication system	2 Weeks
4	Frontend Development	HTML/CSS pages, dashboards	2 Weeks
5	Database Integration	MySQL + AWS RDS connection	1 Week
6	Cloud Deployment	EC2 setup and deployment	1 Week
7	Testing & Optimization	Bug fixing and performance testing	1 Week

7. BENEFITS

1. Faster emergency blood availability
2. Reduced manual dependency
3. Real-time donor matching system
4. Cloud-based scalability and reliability
5. Efficient blood inventory tracking
6. Improved coordination between hospitals and donors

8. PROJECT FLOW

System Architecture Data Flow

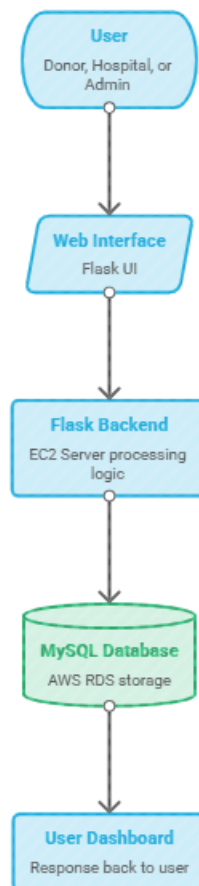


Fig. no 1: Project Flow of BloodBridge: Optimizing Lifesaving Resources using AWS EC2 and RDS

9. SYSTEM ARCHITECTURE DIAGRAM

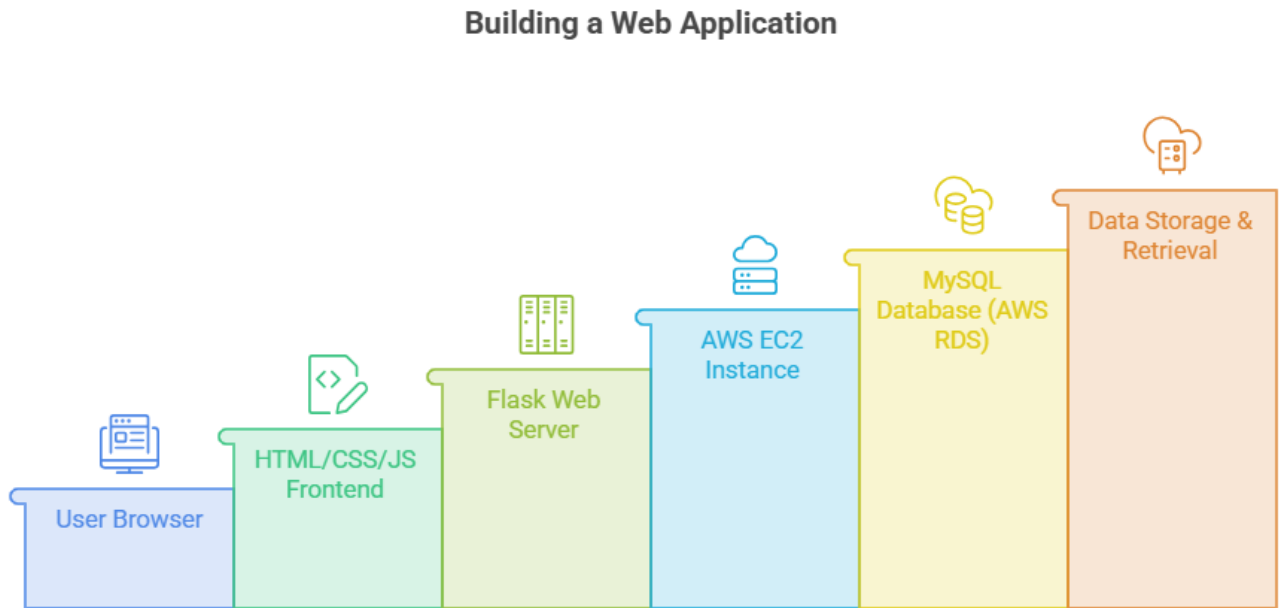


Fig. no 2: System Architecture Diagram of Greenclassify

10. REQUIREMENTS

➤ Hardware Requirements:

1. Minimum 4GB RAM system
2. Stable internet connection
3. AWS account

➤ Software Requirements:

1. Python 3.x
2. Flask framework
3. MySQL database
4. AWS EC2 & RDS
5. VS Code / PyCharm

11. RISKS AND MITIGATIONS

Risk	Mitigation
Database connection failure	Use proper security groups in AWS
Server downtime	Use EC2 monitoring and restart
Data inconsistency	Use validation and constraints
Security threats	Password hashing and session management
Scalability issues	AWS auto-scaling support

12. FUTURE ENHANCEMENTS

1. AI-based blood demand prediction
2. SMS and email notification system
3. Google Maps donor location tracking
4. Mobile application development
5. Automated emergency alert system
6. Chat system between donor and hospital

13. CONCLUSION

BloodBridge is an efficient cloud-based blood management system that reduces delays in emergency situations by connecting donors and hospitals in real time. The use of AWS EC2 and RDS ensures scalability, reliability, and high availability. The system demonstrates practical implementation of full-stack development and cloud computing in healthcare.

14. REFERENCES

1. Amazon Web Services. (2024). AWS EC2 Documentation. Retrieved from <https://aws.amazon.com/ec2/>
2. Amazon Web Services. (2024). Amazon RDS Documentation. Retrieved from <https://aws.amazon.com/rds/>
3. Flask Documentation. (2024). Flask Web Framework. Retrieved from <https://flask.palletsprojects.com/>
4. MySQL. (2023). MySQL Official Documentation. Retrieved from <https://dev.mysql.com/doc/>
5. GitHub. (2024). Version Control System Guide. Retrieved from <https://github.com/>